

Giving AI a Memory

A Journey into Retrieval-Augmented Generation,
Graph RAG, and Autonomous Agents

Based on the lectures of Dr. Chetana Gavankar

Contents

1	Introduction: The Brilliant but Amnesiac Scholar	2
2	Part 1: The Core of RAG	2
2.1	Semantic Search: Understanding Meaning, Not Just Words	2
2.2	The Flow of RAG	3
2.3	Selecting the Right Knowledge: Thresholding vs. Ranking	3
3	Part 2: The Physics of RAG - Budgets and Metrics	3
3.1	The Token Budget	3
3.2	The Science of Chunking (Stride and Overlap)	4
3.3	RAG Quality: Precision and Recall	4
3.4	The Latency Budget	4
4	Part 3: Upgrading RAG with Knowledge Graphs (Graph RAG)	5
4.1	Vector DB vs. Knowledge Graph	5
4.2	The Synergy: Integrating KGs and LLMs	5
4.3	Real World Applications of Graph RAG	6
5	Part 4: The Senses of AI - Multimodal RAG	6
6	Part 5: The Next Evolution - Agentic RAG	6
7	Conclusion & State of the Art	7

1 Introduction: The Brilliant but Amnesiac Scholar

The Isolated Genius

Imagine you hire the most brilliant scholar in the world. They have read every book, Wikipedia page, and article published up until early 2023. They can summarize complex theories, translate languages flawlessly, and write beautiful poetry.

However, there are three massive problems with this scholar:

1. **Knowledge Cutoff:** They have been locked in a windowless room since early 2023. If you ask them about yesterday's news, they are clueless.
2. **Lack of Specificity:** They don't know anything about *your* company's internal HR policies, your proprietary code, or your private databases.
3. **Hallucinations:** Because they have a huge ego, when they don't know the answer, they confidently *make one up*.

This scholar is a metaphor for **Large Language Models (LLMs)** like GPT-4 or Gemini. While they are fantastic at generating human-like text, they suffer from these severe inherent limitations.

How do we fix these flaws? We cannot simply retrain the model every single day—that is computationally impossible and absurdly expensive. The answer is not to make the scholar memorize new books. The answer is to give them a librarian and a filing cabinet of live documents, teaching them to read the relevant files *before* answering.

This strategy is called **Retrieval-Augmented Generation (RAG)**.

2 Part 1: The Core of RAG

Analogy Time: The Open-Book Exam

If standard LLM usage is a closed-book exam where the model relies entirely on its memory, RAG transforms it into an **open-book exam**. When you ask a question, the system first quickly searches the library for the exact pages containing the answer, places those pages on the desk, and says: "*Read these specific pages and answer the question.*"

2.1 Semantic Search: Understanding Meaning, Not Just Words

To give our LLM relevant documents, we need a search mechanism. But traditional keyword-based search is rigid; it looks for exact string matches. If you search for "*cost of laptop repair*," it might completely miss a document titled "*pricing for computer hardware fixes*" because the exact words don't match.

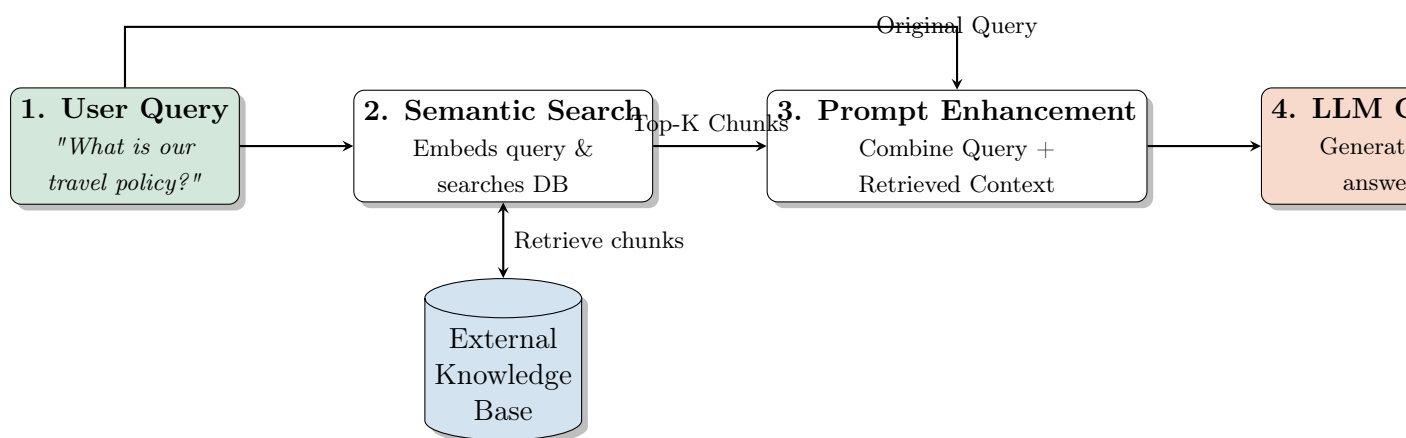
Key Concept: Semantic Search via Embeddings

Semantic search understands the *intent* and *meaning* behind a query. It does this by converting both the query and the knowledge base documents into numerical representations called **Embeddings** using an embedding model.

It places these vectors in a high-dimensional space. The system then finds documents with the closest meaning (shortest mathematical distance), ensuring the most relevant context is found, even if the wording is completely different.

2.2 The Flow of RAG

Here is exactly how a RAG pipeline operates from start to finish:



2.3 Selecting the Right Knowledge: Thresholding vs. Ranking

In RAG, both the query and the document chunks are embedded, and a similarity score is calculated. Higher score = higher semantic relevance. How do we choose what to show the LLM?

- **Ranking:** Select the top- K highest-scoring chunks (e.g., "Give me the top 3").
- **Thresholding:** Select *all* chunks that pass a certain similarity score.

Example: Suppose our similarity scores are: 0.94, 0.88, 0.81, 0.74, 0.69. If our **Threshold** = 0.80, we only pass the first three chunks to the LLM. The rest are discarded to reduce noise. **Note: Ranking and Thresholding are not the same thing!**

3 Part 2: The Physics of RAG - Budgets and Metrics

RAG is constrained by limits. Every LLM has a fixed **Context Window** (a limit on how much text it can process at once). This window is prime real estate. If you shove too many documents into it, the LLM will truncate the text or simply "forget" things. Therefore, retrieval must be carefully budgeted.

3.1 The Token Budget

The Engineering Math: Token Budgeting

Tokens are fragments of words. To figure out how many document chunks we can safely inject, we use this formula:

$$\text{Available Tokens} = \text{Max Context Length} - (\text{System Prompt Tokens} + \text{User Query Tokens})$$

Example:

- Max context length = 8192 tokens
- System prompt = 300 tokens
- User query = 220 tokens

Available = $8192 - (300 + 220) = \mathbf{7672 \text{ tokens}}$.

If each retrieved chunk is exactly 512 tokens, the maximum number of chunks we can include is: $\lfloor 7672/512 \rfloor = \mathbf{14 \text{ chunks}}$.

Why floor is used: Exceeding the context limit is absolutely not allowed.

3.2 The Science of Chunking (Stride and Overlap)

When we break a massive PDF into small chunks, we run the risk of cutting a sentence in half, destroying its meaning. To solve this, we use **Chunk Overlap**.

The Engineering Math: Calculating Stride and Chunk Count

Effective Stride is how far the "window" moves forward when cutting the next chunk.

Stride = Chunk Size – Overlap

Example:

- Total Document Length = 18,200 tokens
- Chunk Size = 600 tokens
- Overlap = 120 tokens

Stride = $600 - 120 = \mathbf{480 \text{ tokens}}$.

To find out how many chunks this document will produce: $\lceil 18200/480 \rceil = \mathbf{38 \text{ chunks}}$.

Why ceiling is used: Partial chunks at the end still count as a full chunk in the database.

3.3 RAG Quality: Precision and Recall

Retrieval quality determines generation quality ("Garbage in, garbage out").

- **Precision:** $\frac{\text{Number of relevant retrieved chunks}}{\text{Total retrieved chunks}}$. *Precision controls noise.*
- **Recall:** $\frac{\text{Number of relevant retrieved chunks}}{\text{Total relevant chunks in the corpus}}$. *Recall controls coverage.*

Example: If there are 20 total relevant documents in the corpus, and we retrieve 13 of them, our Recall is $13/20 = \mathbf{0.65}$. This means 65% of the required knowledge is available to the LLM. Higher recall generally improves the completeness of generated answers.

3.4 The Latency Budget

Real-time RAG systems must respond within a strict time budget.

Total Latency = Embedding Time + Retrieval Time + (Re-ranking Time) + Generation Time

The Engineering Math: Pipeline Latency

Can we afford to add an advanced Re-ranking step to our pipeline?

- Total Allowed Time = 2500 ms
- Embedding = 150 ms
- Retrieval = 420 ms
- Generation = 1600 ms

Time Used = $150 + 420 + 1600 = \mathbf{2170 \text{ ms}}$.
 Remaining Budget = $2500 - 2170 = \mathbf{330 \text{ ms}}$.
 If our re-ranking algorithm requires 300 ms, **yes**, it can be added without exceeding the budget!

4 Part 3: Upgrading RAG with Knowledge Graphs (Graph RAG)

Traditional RAG stores text chunks in a **Vector Database** (e.g., Pinecone, FAISS, Weaviate). This is great for unstructured text, but it struggles with complex, logical questions.

The Limitation of Vectors

If you ask a Vector DB, *"Who is the CEO of my company?"*, it works perfectly. It finds a chunk saying "John Doe is the CEO."
 But what if you ask, **"Which board meetings in the last twelve months had at least two members abstain from a vote?"** A Vector DB relies on finding text that "sounds" similar to the query. It will likely return incomplete or irrelevant results. A **Knowledge Graph (KG)**, however, maps entities and explicit relationships (e.g., Member $\rightarrow$ Abstains $\rightarrow$ Meeting $\rightarrow$ Date). It can traverse these connections to give a mathematically precise answer.

4.1 Vector DB vs. Knowledge Graph

Feature	Traditional RAG (Vector)	Graph RAG (KG)
Data Format	Unstructured Text	Structured Graph (Nodes/Edges)
Retrieval Method	Dense Vector Search	Traversal / Querying (e.g., Cypher)
Reasoning	Limited (Single-hop)	Enhanced (Multi-hop)
Explainability	Low (Black box vectors)	High (Clear pathways)
Integration	Moderate Complexity	High Complexity
Use Cases	General Q&A	Domain-Specific, Complex Logic

4.2 The Synergy: Integrating KGs and LLMs

KGs and LLMs are the perfect couple. KGs provide factual, structured, explicit, and interpretable knowledge. LLMs provide vast general knowledge, generalizability, and natural language interfaces.

There are three main synergy approaches:

- KG-Enhanced LLMs (Retrieval):** The LLM uses a Knowledge Retriever to fetch verified facts from the KG (e.g., Obama $\rightarrow$ BornIn $\rightarrow$ Honolulu and Honolulu $\rightarrow$ LocatedIn $\rightarrow$ USA) to output "USA".
- LLM-Augmented KGs (Extraction):** We use an LLM (like GPT-3) to read raw medical papers and automatically construct the Knowledge Graph.
 - Example:* GPT-3 reads: "EGF up-regulates miR-31". It extracts the triplet (EGF, upregulate, miR-31) and pushes it to a Neo4j Graph DB.

3. **Synergized LLMs + KGs:** A continuous loop where data representation learning and neural-symbolic reasoning work together in applications like Search Engines or Dialogue Systems.

4.3 Real World Applications of Graph RAG

- **Personalized Medicine:** Integrating KGs (like Hetionet, KEGG) with LLMs for accurate medical Q&A. Example: Systems evaluating COVID treatments like remdesivir.
- **Social Good (KGSG):** Applying Knowledge Graphs to the Human Trafficking domain to build entity-centric search engines.
- **Explainable AI (XAI):** In DoD (Department of Defense), Medicine, or Finance, AI must answer: *"Why did you do that? When can I trust you?"* KGs provide the transparent logic trail required for XAI.

5 Part 4: The Senses of AI - Multimodal RAG

Our enterprise knowledge isn't just text. It's stored in:

- **Images:** Product photos, diagrams in manuals, satellite imagery.
- **Documents:** PDFs containing charts, tables, and complex layouts.
- **Videos & Audio:** How-to tutorials, product demos, customer support calls.

Standard RAG is blind to this. It cannot answer: *"What were the Q3 revenue trends based on this chart in the annual report?"*

Key Concept: Multimodal RAG

Multimodal RAG converts *multiple data types* into a shared vector space. A specialized **Multimodal Embedding** model looks at a picture of a dog and converts it into a vector. It places that vector right next to the text vector for the word "dog" in a shared "meaning space."

Now, a user can upload an image (e.g., a broken part), the system retrieves the relevant text manual, and a powerful Multimodal LLM (like **Gemini** or GPT-4o) synthesizes an answer from both!

6 Part 5: The Next Evolution - Agentic RAG

We are moving from passive RAG to **Agentic RAG**. In standard RAG, the system is a linear pipeline: Retrieve → Generate.

In Agentic RAG, the LLM acts as an autonomous **Agent**. Instead of just answering, it:

- **Plans (Sets Subgoals):** Query: *"Compare nuclear vs. solar output globally in 2024."* The Agent decomposes this into sub-queries.
- **Uses Tools (Executor):** It decides to search the web for solar data, query a KG for nuclear data, and use a calculator to standardize the units.
- **Iterates (Memory):** It uses a "scratchpad" to store past actions. If the first search is bad, it refines the search and tries again before synthesizing the final answer with citations.

7 Conclusion & State of the Art

The integration of Knowledge Graphs with LLMs offers a promising avenue to enhance AI by combining structured knowledge with generative capabilities.

Current **State of the Art** models include:

- **DRAGON:** Jointly pre-trains on text and KGs for complex reasoning.
- **KG-RAG:** Integrates KGs with RAG (focusing heavily on financial data).
- **GFM-RAG:** Introduces a graph foundation model for multi-hop QA.
- **GNN-RAG:** Combines Graph Neural Networks with RAG.

The future of RAG is **Agentic** and **Multimodal**. By mastering these paradigms, we turn LLMs from isolated, hallucinating scholars into highly reliable, deeply knowledgeable, autonomous enterprise assistants.